

# Bölüm 8 - Yazılım Testi

# Konular

- Geliştirme testi
- Test güdümlü geliştirme
- Dağıtım testi
- Kullanıcı testi

# Program Testi

- Testin amacı, bir programın yapması amaçlanan şeyi yaptığını göstermek ve program kullanılmadan önce hatalarını keşfetmektir.
- Yazılımı test ettiğinizde, yapay verileri kullanarak bir program çalıştırırsınız.
- Programın işlevsel olmayan özellikleri hakkında hatalar, anormallikler veya bilgiler için test çalıştırmasının sonuçlarını kontrol edersiniz.
- Hataların yokluğunu DEĞİL, varlığını ortaya çıkarabilir.
- Test, statik doğrulama tekniklerini de içeren daha genel bir doğrulama ve onaylama sürecinin bir parçasıdır.

# Program Testinin Hedefleri

- Yazılımın gereksinimlerini karşıladığını geliştiriciye ve müşteriye göstermek.
  - Özel yazılım için, gereksinimler dökümanındaki her gereksinim için en az bir test olması gerektiği anlamına gelir. Genel yazılım ürünleri için, tüm sistem özellikleri için testlerin yanı sıra ürün sürümüne dahil edilecek bu özelliklerin kombinasyonlarının yapılması gerektiği anlamına gelir.
- Yazılımın davranışının yanlış, istenmeyen veya tanımına uymadığı durumları keşfetmek.
  - Hata testi, sistem çökmeleri, diğer sistemlerle istenmeyen etkileşimler, yanlış hesaplamalar ve veri bozulması gibi istenmeyen sistem davranışlarını ortadan kaldırmakla ilgilidir.

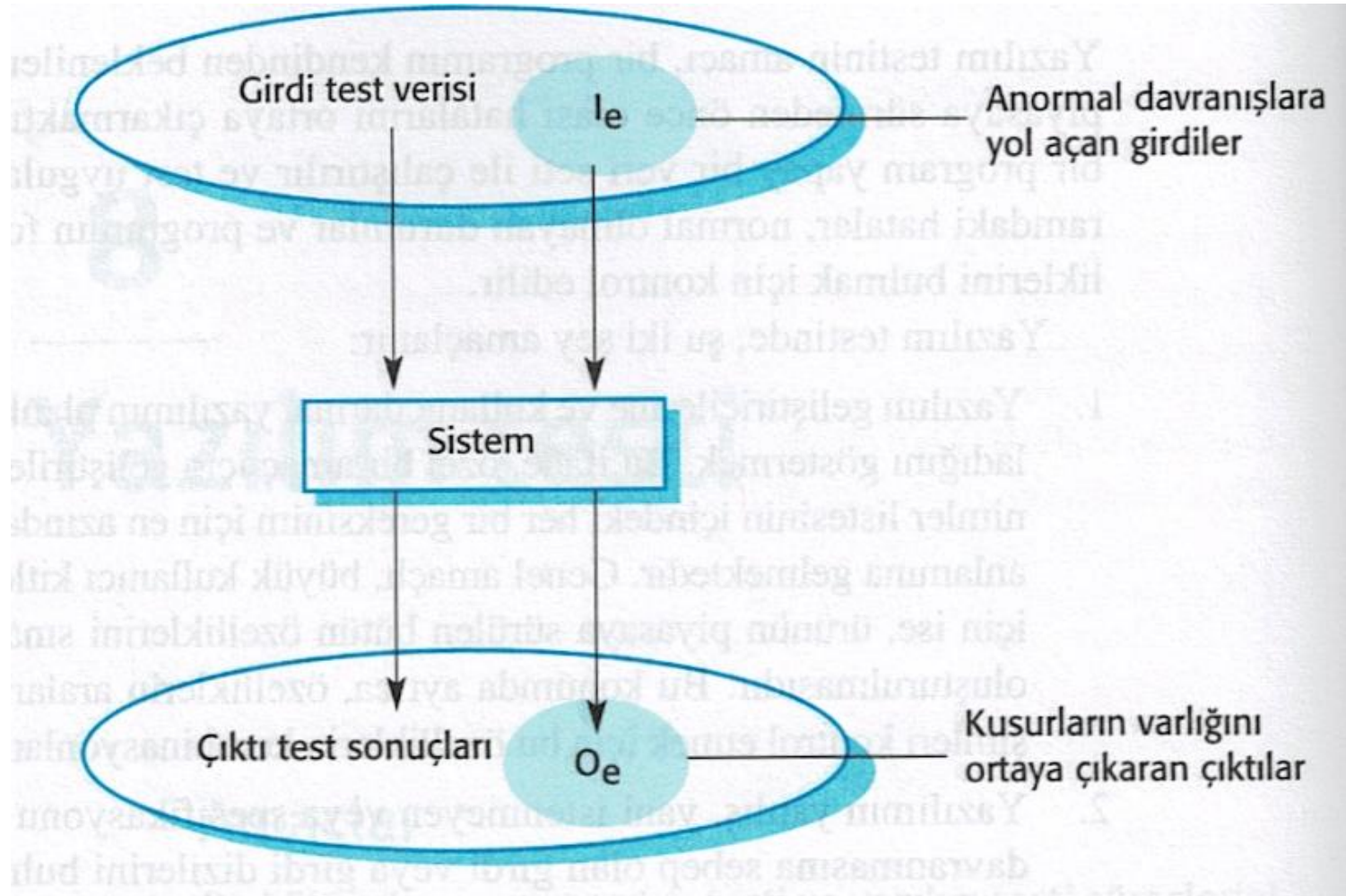
# Onaylama ve Hata Testi

- İlk hedef doğrulama testine götürür
  - Sistemin beklenen kullanımını yansıtan belirli bir dizi test senaryosunu kullanarak doğru şekilde çalışmasını beklersiniz.
- İkinci hedef, kusur testine götürür
  - Test senaryoları, hataları ortaya çıkarmak için tasarlanmıştır. Hata testindeki test senaryoları kasıtlı olarak belirsiz olabilir ve sistemin normal olarak nasıl kullanıldığını yansıtması gerekmez.

# Test Sürecinin Amaçları

- Onaylama testi
  - Geliştiriciye ve sistem müşterisine yazılımın gereksinimlerini karşıladığını göstermek için
  - Başarılı bir test, sistemin amaçlandığı gibi çalıştığını gösterir.
- Hata testi
  - Yazılımda çalışmasının yanlış olduğu veya tanımına uygun olmayan hataları keşfetmek
  - Başarılı bir test, sistemin hatalı çalışmasına neden olan ve bu nedenle sistemdeki bir kusuru ortaya çıkaran bir testtir.

# Program Testinin Girdi-Çıktı Modeli



# Doğrulama ve Onaylama

- Doğrulama:  
«Ürünü doğru geliştiriyor muyuz»
- Yazılım, teknik özelliklerine uygun olmalıdır.
- Onaylama:  
«Doğru ürünü mü üretiyoruz»
- Yazılım, kullanıcının gerçekten ihtiyaç duyduğu şeyi yapmalıdır.



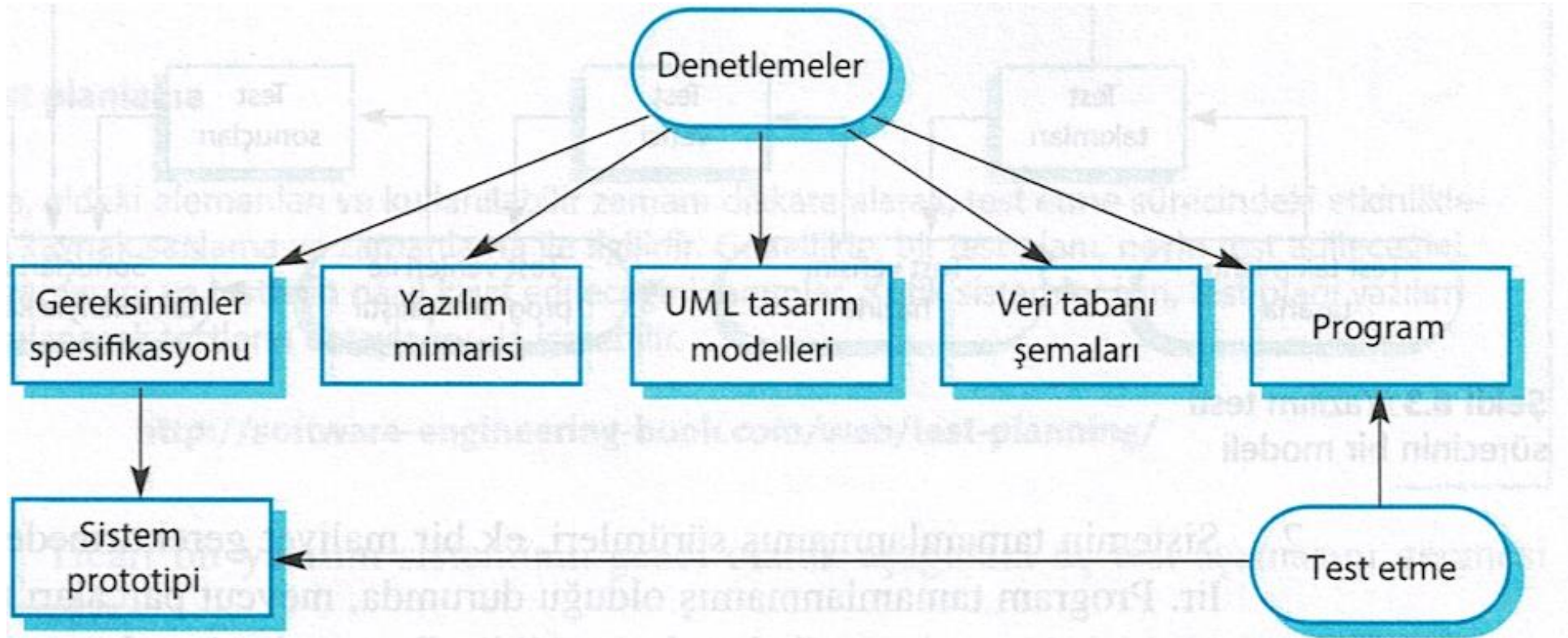
# Doğrulama ve Onaylama Güveni

- Doğrulama ve onaylamanın amacı, sistemin "amaca uygun" olduğuna dair güven oluşturmaktır.
- Sistemin amacına, kullanıcı beklentilerine ve pazarlama ortamına bağlıdır
  - Yazılım amacı
    - ❑ Güven seviyesi, yazılımın bir kuruluş için ne kadar kritik olduğuna bağlıdır.
  - Kullanıcı beklentileri
    - ❑ Kullanıcıların belirli yazılım türlerinden beklentileri düşük olabilir.
  - Pazarlama ortamı
    - ❑ Bir ürünü pazara erken sürmek, programdaki kusurları bulmaktan daha önemli olabilir.

# Denetlemeler ve Test Etme

- Yazılım denetlemeleri - Sorunları keşfetmek için statik sistem gösteriminin analizi ile ilgilidir (statik doğrulama)
  - Araç tabanlı döküman ve kod analizi ile tamamlanabilir.
- Yazılım test etme - Ürün davranışını uygulamak ve gözlemlemekle ilgili (dinamik doğrulama)
  - Sistem test verisi ile çalıştırılır ve operasyonel davranışı gözlemlenir.

# Denetlemeler ve Test Etme



# Yazılım Denetlemeleri

- Bunlar, anormallikleri ve hataları keşfetmek amacıyla kaynak gösterimini inceleyen insanları içerir.
- Denetimler bir sistemin çalışması gerekmeden uygulamadan önce kullanılabilir.
- Sistemin herhangi bir temsiline uygulanabilir (gereksinimler, tasarım, konfigürasyon verileri, test verileri, vb.).
- Program hatalarını keşfetmek için etkili bir teknik oldukları gösterilmiştir.

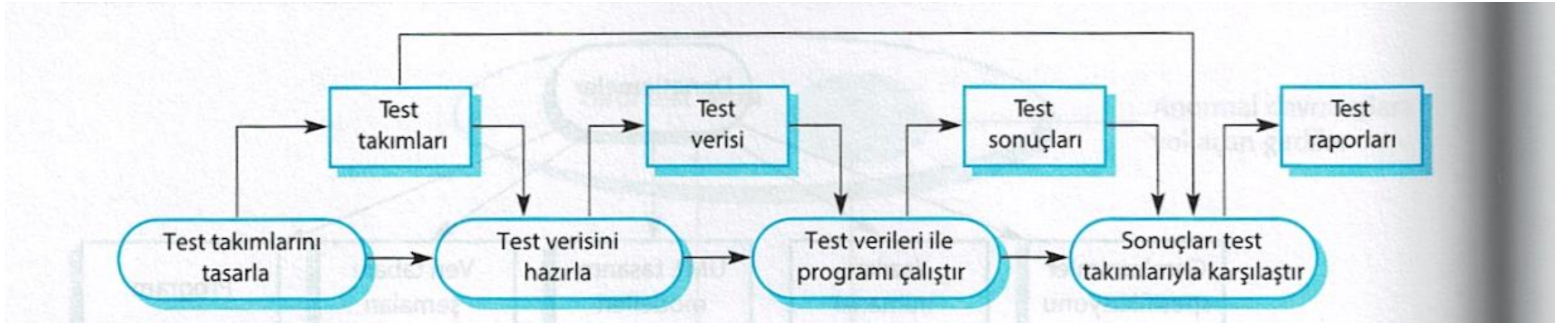
# Denetlemelerin Avantajları

- Test sırasında, hatalar diğer hataları maskeleyebilir (gizleyebilir). Denetleme statik bir süreç olduğu için, hatalar arasındaki etkileşimlerle ilgilenmenize gerek yoktur.
- Bir sistemin eksik sürümleri ek maliyet olmadan denetlenebilir. Bir program tamamlanmamışsa, mevcut parçaları test etmek için özel test donanımları geliştirmeniz gerekir.
- Bir denetleme, program hatalarını aramanın yanı sıra, bir programın standartlara uyum, taşınabilirlik ve sürdürülebilirlik gibi daha geniş kalite özelliklerini de dikkate alabilir.

# Denetlemeler ve Test Etme

- Denetlemeler ve testler birbirini tamamlayıcı niteliktedir ve doğrulama tekniklerine karşı değildir.
- Her ikisi de doğrulama ve onaylama işlemi sırasında kullanılmalıdır.
- Denetlemeler, bir tanıma uygunluğu kontrol edebilir, ancak müşterinin gerçek gereksinimlerine uygunluğu kontrol edemez.
- Denetlemeler, performans, kullanılabilirlik vb. Gibi işlevsel olmayan özellikleri kontrol edemez.

# Yazılım Testi Sürecinin Bir Modeli



# Test Aşamaları

- Geliştirme testi - Sistem, geliştirme sırasında hataları ve kusurları keşfetmek için test edilir.
- Dağıtım testi - Ayrı bir test ekibinin, kullanıcılara sunulmadan önce sistemin eksiksiz bir sürümünü test ettiği durumlarda.
- Kullanıcı testi - Bir sistemin kullanıcıları veya potansiyel kullanıcıları sistemi kendi ortamlarında test ettiğinde.



# **Geliştirme Testi**

# Geliştirme Testi

- Geliştirme testleri, sistemi geliştiren ekip tarafından gerçekleştirilen tüm test faaliyetlerini içerir.
  - Birim testi - Bireysel program birimlerinin veya nesne sınıflarının test edildiği yer. Birim testi, nesnelerin veya yöntemlerin işlevselliğini test etmeye odaklanmalıdır.
  - Bileşen testi - Tümleşik bileşenler oluşturmak için birkaç bağımsız birimin entegre edildiği yer. Bileşen testi, bileşen arayüzlerini test etmeye odaklanmalıdır.
  - Sistem testi - Bir sistemdeki bileşenlerin bir kısmının veya tümünün entegre olduğu ve sistemin bir bütün olarak test edildiği yer. Sistem testi, bileşen etkileşimlerini test etmeye odaklanmalıdır.

# Birim Testi

- Birim testi, tek tek bileşenlerin ayrı ayrı test edilmesi sürecidir.
- Hata testi sürecidir.
- Birimler şunlar olabilir:
  - Bir nesne içindeki bireysel işlevler veya yöntemler
  - Çeşitli niteliklere ve yöntemlere sahip nesne sınıfları
  - İşlevselliklerine erişmek için kullanılan tanımlı arayüzlere sahip tümleşik bileşenler.

# Nesne Sınıf Testi

- Bir sınıfın eksiksiz test kapsamı şunları içerir:
  - Bir nesneyle ilişkili tüm işlemleri test etme
  - Tüm nesne niteliklerini ayarlama ve sorgulama
  - Nesneyi tüm olası durumlarda kullanmak.
- Kalıtım, test edilecek bilgi yerelleştirilmediğinden nesne sınıfı testlerinin tasarlanmasını zorlaştırır.

# Hava İstasyonu Nesnesinin Arayüzü

| HavaDurumuIstasyonu                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tanitici                                                                                                                                                             |
| havaDurumuRaporla ()<br>durumRaporla ()<br>gucKoruma(aygitlar)<br>uzakKontrol(komutlar)<br>yenidenYapilandir(komutlar)<br>yenidenBaslat(aygitlar)<br>kapat(aygitlar) |

# Hava İstasyonu Testi

- havaDurumuRaporla gibi nesneler için kalibrasyon, test, başlatma ve kapatma test senaryoları tanımlamanız gerekir.
- Bir durum modeli kullanarak, test edilecek durum geçişleri dizilerini ve bu geçişlere neden olacak olay dizilerini tanımlayın
- Örneğin:
  - Kapat → Çalışma → Kapat
  - Yapılandırma → Çalışma → Test → İletilme → Çalışma
  - Çalışma → Toplama → Çalışma → Özetleme → İletilme → Çalışma

# Otomatik Test

- Mümkün olduğunda, testlerin manuel müdahale olmadan çalıştırılması ve kontrol edilmesi için ünite testi otomatikleştirilmelidir.
- Otomatik birim testinde, program testlerinizi yazmak ve çalıştırmak için bir test otomasyon çerçevesi (JUnit gibi) kullanırsınız.
- Birim testi çerçeveleri, belirli test senaryoları oluşturmak için genişlettiğiniz genel test sınıfları sağlar. Daha sonra uyguladığınız tüm testleri çalıştırabilir ve genellikle bazı GUI'ler aracılığıyla testlerin aksi yöndeki başarısını rapor edebilirler.

# Otomatik Test Bileşenleri

- Sistemi test senaryosu ile başlattığınız bir kurulum bölümü, yani girdiler ve beklenen çıktılar.
- Test edilecek nesneyi veya yöntemi çağırdığınız bir çağrı bölümü.
- Çağrının sonucunu beklenen sonuçla karşılaştırdığınız bir onaylama bölümü. İddia doğru olarak değerlendirilirse, test başarılı olmuştur. Yanlışsa, başarısız olmuştur.



# Birim Test Durumlarını Seçme

- Test senaryoları, beklendiği gibi kullanıldığında, test ettiğiniz bileşenin yapması gerekeni yaptığını göstermelidir.
- Bileşende kusurlar varsa, bunlar test senaryoları ile ortaya çıkarılmalıdır.
- Bu, 2 tür birim test senaryosuna yol açar:
  - Bunlardan ilki, bir programın normal işleyişini yansıtmalı ve bileşenin beklendiği gibi çalıştığını göstermelidir.
  - Diğer tür test senaryosu, yaygın sorunların ortaya çıktığı yerdeki test deneyimine dayanmalıdır. Bunların düzgün şekilde işlendiğini ve bileşeni çökertmediğini kontrol etmek için anormal girdiler kullanmalıdır.

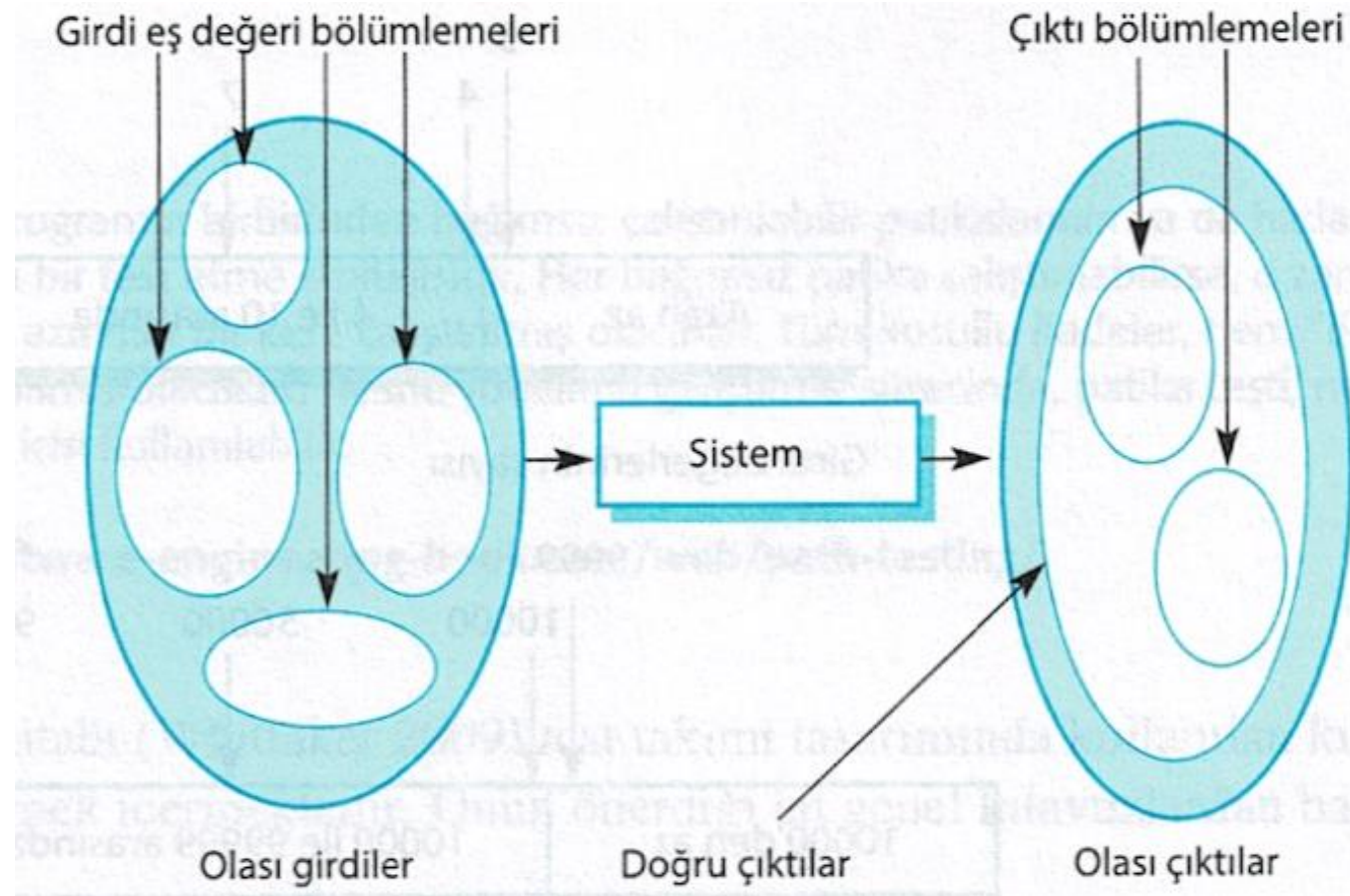
# Test Stratejileri

- Bölünme testi - Ortak özelliklere sahip olan ve aynı şekilde işlenmesi gereken girdi gruplarını belirlediğiniz yer.
  - Bu grupların her biri içinden test seçmelisiniz.
- Kılavuzlara dayalı test - Test senaryolarını seçmek için test kılavuzlarını kullandığınız yer.
  - Bu kılavuzlar, programcıların bileşenleri geliştirirken sıklıkla yaptıkları hata türlerine ilişkin önceki deneyimlerini yansıtır.

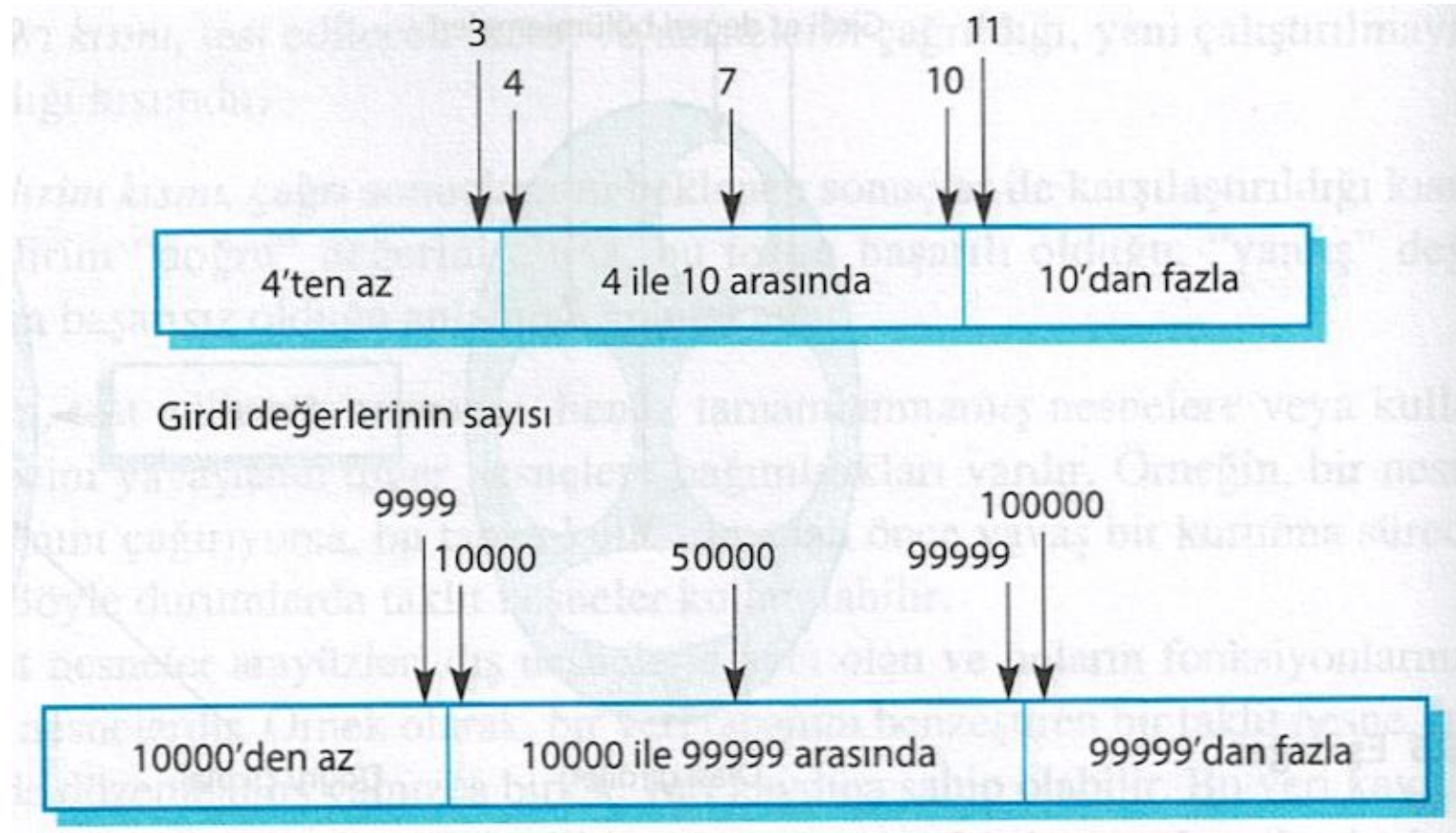
# Bölünme Testi

- Girdi verileri ve çıktı sonuçları genellikle bir sınıfın tüm üyelerinin ilişkili olduğu farklı sınıflara girer.
- Bu sınıfların her biri, programın her sınıf üyesi için eşdeğer bir şekilde davrandığı bir eşdeğerlik bölümü veya etki alanıdır.
- Her bölümden test senaryoları seçilmelidir.

# Eş Değer Bölünme



# Eş Değer Bölünmeler



# Test Kılavuzları (diziler)

- Yalnızca tek bir değere sahip dizileri olan test yazılımı.
- Farklı testlerde farklı boyutlarda diziler kullanın.
- Dizinin ilk, orta ve son öğelerine erişilecek şekilde testleri türetin.
- Sıfır uzunluktaki dizilerle test edin.

# Genel Test Kılavuzları

- Sistemi tüm hata mesajlarını oluşturmaya zorlayan girdileri seçin
- Girdi arabelliklerinin taşmasına neden olan girdiler tasarla
- Aynı girdiyi ve girdi dizilerini çok defa uygula
- Geçersiz çıktıların üretilmesini sağla
- Hesaplama sonuçlarının çok büyük ya da çok küçük olmasını sağla

# Bileşen Testi

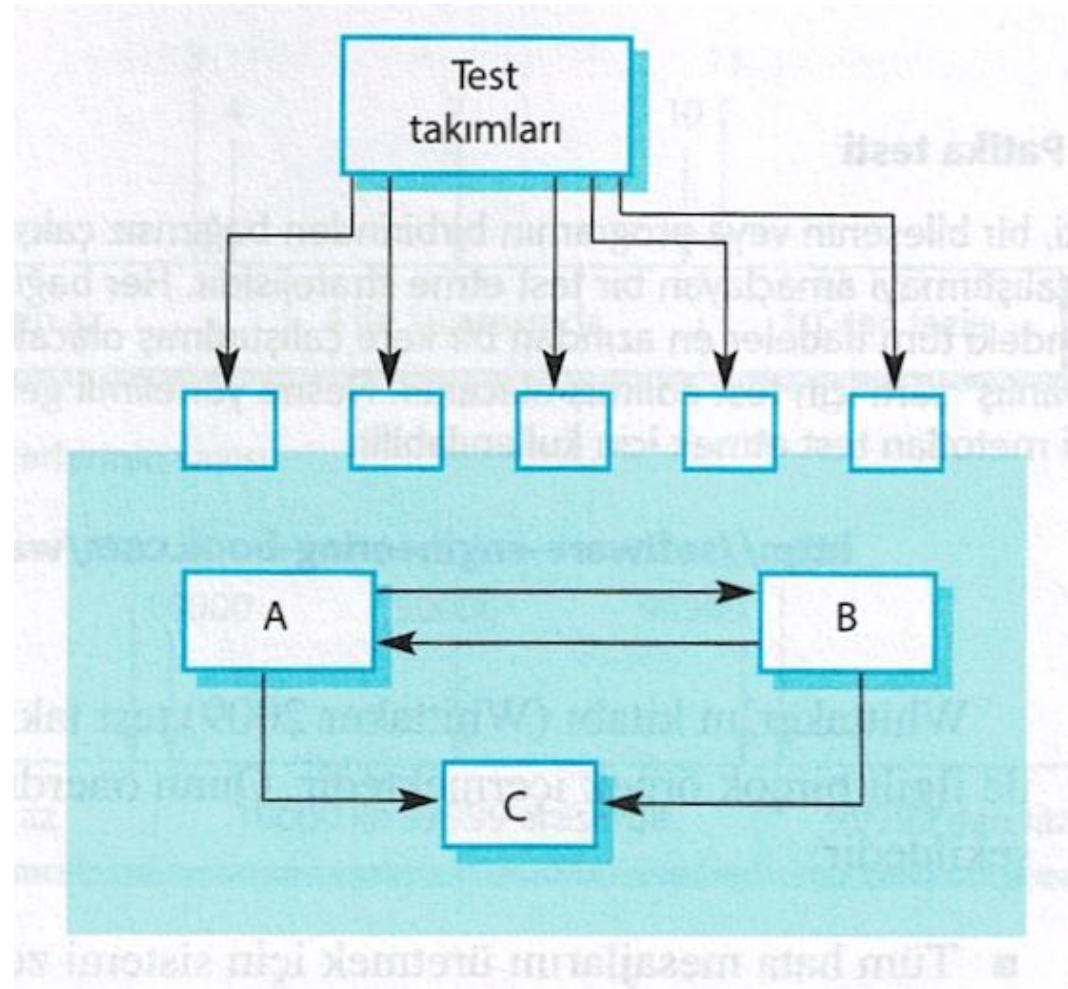
- Yazılım bileşenleri genellikle birkaç etkileşimli nesneden oluşan tümleşik bileşenlerdir.
  - Örneğin, meteoroloji istasyonu sisteminde, yeniden yapılandırma bileşeni, yeniden yapılandırmanın her bir yönü ile ilgilenen nesneleri içerir.
- Bu nesnelerin işlevselliğine, tanımlı bileşen arayüzü üzerinden erişirsiniz.
- Bu nedenle, tümleşik bileşenlerin test edilmesi, bileşen arayüzünün özelliklerine göre davrandığını göstermeye odaklanmalıdır.
  - Bileşen içindeki ayrı nesneler üzerindeki birim testlerinin tamamlandığını varsayabilirsiniz.



# Arayüz Testi

- Amaç, arayüz hataları veya arayüzlerle ilgili geçersiz varsayımlar nedeniyle oluşan hataları tespit etmektir.
- Arayüz türleri
  - **Parametre arayüzleri** - Bir yöntem veya prosedürden diğerine geçen veriler.
  - **Paylaştırılmış bellek arayüzleri** - Bellek bloğu, prosedürler veya işlevler arasında paylaşılır.
  - **Prosedürel arayüzler** - Alt sistem, diğer alt sistemler tarafından çağrılacak bir dizi prosedürü kapsamaktadır.
  - **Mesaj geçiş arayüzleri** - Alt sistemler diğer alt sistemlerden hizmet talep eder

# Arayüz Testi



# Arayüz Hataları

- Arayüzün yanlış kullanımı
  - Çağrı yapan bir bileşen başka bir bileşeni çağırır ve onun arayüzünü kullanırken bir hata yapar, örn. parametreler yanlış sırada.
- Arayüz yanlış anlaşılması
  - Çağrı yapan bir bileşen, çağırdığı bileşenin arayüzünün tanımını yanlış anlayıp, onun hakkında yanlış varsayımlar yapar.
- Zamanlama hataları
  - Çağrı yapan ve çağrılan bileşen farklı hızlarda çalışır ve güncel olmayan bilgilere erişilir.

# Arayüz Testi Kuralları

- Testleri, çağrılan bir prosedürün parametrelerinin aralıklarının en uç noktalarında olması için tasarlayın.
- İşaretçi parametrelerini her zaman boş işaretçilerle test edin.
- Bileşenin başarısız olmasına neden olan tasarım testleri.
- Mesaj geçirme sistemlerinde stres testini kullanın.
- Paylaşılan bellek sistemlerinde, bileşenlerin etkinleştirilme sırasını değiştirin.

# Sistem Testi

- Geliştirme sırasında sistem testi, sistemin bir sürümünü oluşturmak için bileşenleri entegre etmeyi ve ardından entegre sistemi test etmeyi içerir.
- Sistem testinin odak noktası, bileşenler arasındaki etkileşimleri test etmektir.
- Sistem testi, bileşenlerin uyumlu olup olmadığını, doğru etkileşimde bulunup bulunmadığını ve doğru verileri doğru zamanda arayüzleri üzerinden aktardığını kontrol eder.
- Sistem testi, bir sistemin ortaya çıkan davranışını test eder.

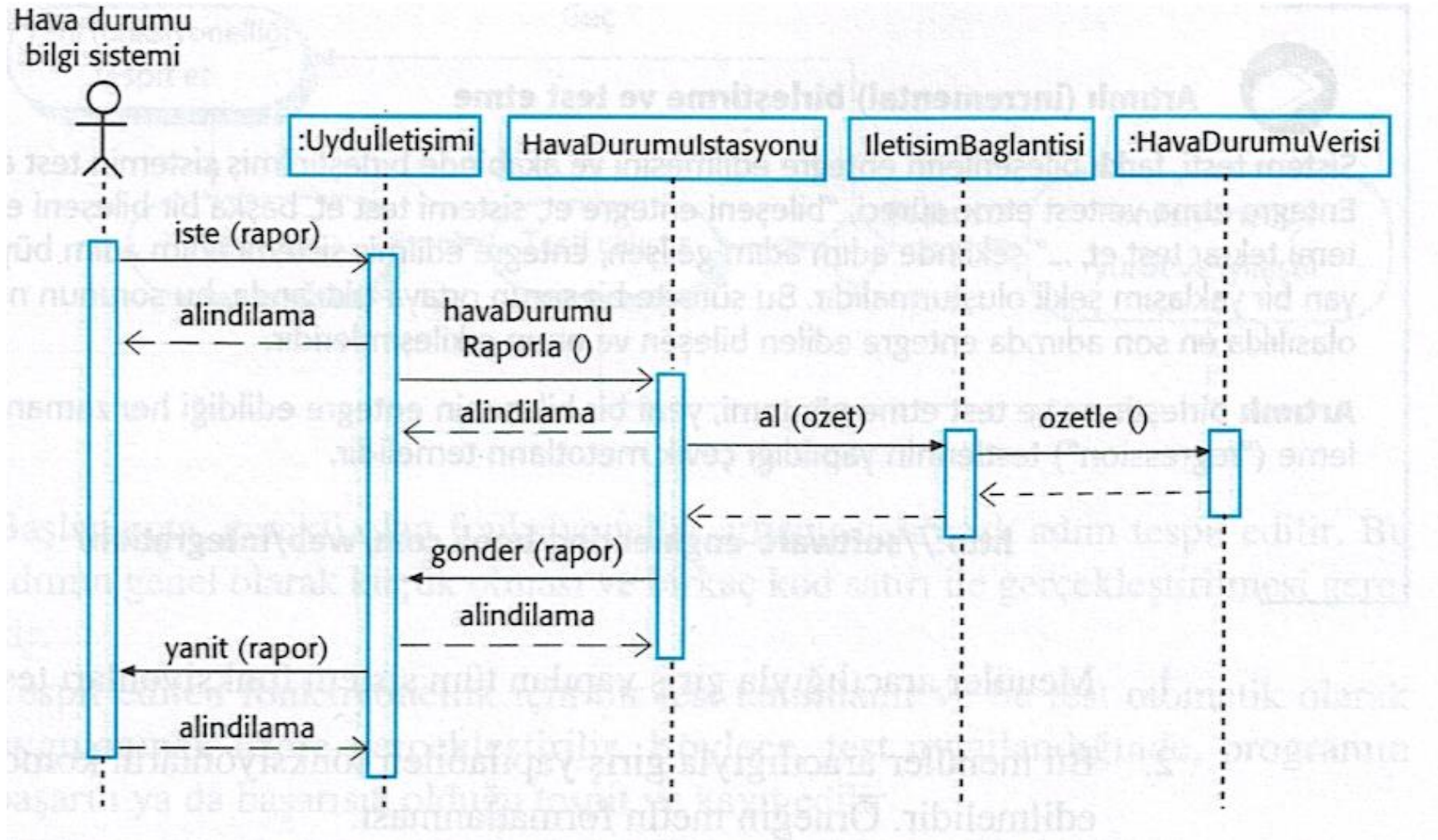
# Sistem ve Bileşen Testi

- Sistem testi sırasında, ayrı ayrı geliştirilen ve kullanıma hazır sistemlerdeki yeniden kullanılabilir bileşenler yeni geliştirilen bileşenlerle entegre edilebilir. Tüm sistem daha sonra test edilir.
- Farklı ekip üyeleri veya alt ekipler tarafından geliştirilen bileşenler bu aşamada entegre edilebilir. Sistem testi, bireysel bir süreçten ziyade toplu bir süreçtir.
  - Bazı şirketlerde, sistem testi, tasarımcıların ve programcıların katılımı olmaksızın ayrı bir test ekibini içerebilir.

# Kullanım Durum Testi

- Sistem etkileşimlerini tanımlamak için geliştirilen kullanım durumları, sistem testi için bir temel olarak kullanılabilir.
- Her bir kullanım senaryosu genellikle birkaç sistem bileşenini içerir, bu nedenle kullanım senaryosunun test edilmesi bu etkileşimleri gerçekleştirmeye zorlar.
- Kullanım senaryosu ile ilişkili sıra diyagramları, test edilen bileşenleri ve etkileşimleri belgeler.

# Hava Veri Dizisi Toplama Grafiği





# Sıra Diyagramından Türetilen Test Senaryoları

- Bir rapor talebinin girdisi, ilişkili bir alındı bilgisine sahip olmalıdır. Nihayetinde talepten bir rapor elde edilmelidir.
  - Raporun doğru şekilde organize edildiğini kontrol etmek için kullanılabilecek özet veriler oluşturmalsınız.
- Havaİstasyonu'na bir rapor için girdi talebi, oluşturulan özet bir raporla sonuçlanır.
  - SatComms testi için hazırladığınız özete karşılık gelen ham veriler oluşturularak ve Havaİstasyonu nesnesinin bu özeti doğru şekilde oluşturup oluşturmadığını kontrol ederek test edilebilir. Bu ham veriler, HavaVeri nesnesini test etmek için de kullanılır.

# Test Politikaları

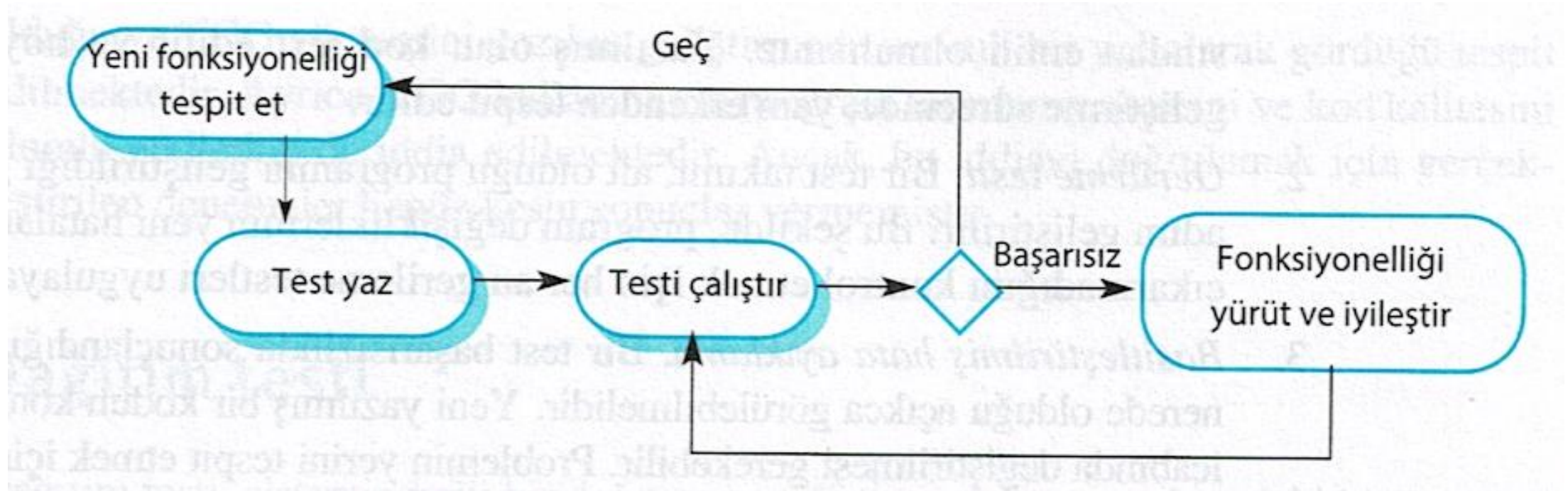
- Kapsamlı sistem testi imkansız olduğundan, gerekli sistem testi kapsamını tanımlayan test politikaları geliştirilebilir.
- Test politikalarına örnekler:
  - Menüler aracılığıyla erişilen tüm sistem işlevleri test edilmelidir.
  - Aynı menüden erişilen işlev kombinasyonları (ör. metin biçimlendirme) test edilmelidir.
  - Kullanıcı girişi sağlandığında, tüm işlevler hem doğru hem de yanlış girdiyle test edilmelidir.

# **Test Gdml Geliřtirme**

# Test Gdml Geliřtirme

- Test Gdml Geliřtirme (TGG), test etmenin ve kod geliřtirmenin dnřml olarak uygulandığı bir program geliřtirme yaklařımıdır.
- Testler koddan nce yazılır ve testleri "geçmek", geliřtirmenin kritik itici gcdr.
- Bir test ile birlikte arttırımsal olarak kod geliřtirirsiniz. Geliřtirdiğiniz kod, testi geene kadar bir sonraki ařamaya geemezsiniz.
- TGG, U Programlama (Extreme Programming) gibi evik yntemlerin bir parası olarak tanıtıldı. Ancak plan odaklı geliřtirme srelerinde de kullanılabilir.

# Test Gdml Geliřtirme



# TGG Süreç Faaliyetleri

- Gerekli olan işlevsellik artışını belirleyerek başlayın. Bu normalde küçük ve birkaç satır kodda uygulanabilir olmalıdır.
- Bu işlevsellik için bir test yazın ve bunu otomatik bir test olarak uygulayın.
- Uygulanan diğer tüm testlerle birlikte testi çalıştırın. Başlangıçta, işlevselliği uygulamamışsınızdır, bu nedenle yeni test başarısız olacaktır.
- İşlevselliği uygulayın ve testi yeniden çalıştırın.
- Tüm testler başarıyla çalıştıktan sonra, bir sonraki işlevsellik parçasını uygulamaya geçersiniz.

# Test GÜdümlü Geliştirmenin Yararları

- Kod kapsam
  - Yazdığınız her kod bölümünün en az bir ilişkili testi vardır, bu nedenle yazılan tüm kodda en az bir test vardır.
- Gerileme (Regression) testi
  - Bir program geliştirildikçe arttırımsal olarak bir regresyon testi paketi geliştirilir.
- Basitleştirilmiş hata ayıklama
  - Bir test başarısız olduğunda, sorunun nerede olduğu belli olmalıdır. Yeni yazılan kodun kontrol edilmesi ve değiştirilmesi gerekiyor.
- Sistem dökümantasyonu
  - Testlerin kendileri, kodun ne yapması gerektiğini açıklayan bir belge türüdür.

# Gerileme Testi

- Gerileme testi, değişikliklerin daha önce çalışan kodun 'bozulmadığını' kontrol etmek için sistemi test ediyor.
- Manuel test sürecinde, gerileme testi pahalıdır, ancak otomatik test ile basit ve kolaydır. Tüm testler, programda her değişiklik yapıldığında yeniden çalıştırılır.
- Testler, değişiklik yapılmadan önce 'başarıyla' çalıştırılmalıdır.



# **Dağıtım Testi**

# Dağıtım Testi

- Dağıtım testi, geliştirme ekibinin dışında kullanılması amaçlanan bir sistemin belirli bir sürümünü test etme sürecidir.
- Dağıtım testi sürecinin birincil amacı, sistemin tedarikçisini kullanım için yeterince iyi olduğuna ikna etmektir.
  - Bu nedenle dağıtım testi, sistemin belirtilen işlevselliğini, performansını ve güvenilirliğini sağladığını ve normal kullanım sırasında başarısız olmadığını göstermelidir.
- Dağıtım testi, genellikle testlerin yalnızca sistem tanımından türetildiği bir kara kutu test sürecidir.

# Dağıtım Testi ve Sistem Testi

- Dağıtım testi, bir sistem testi biçimidir.
- Önemli farklılıklar:
  - Sistem geliştirmeye dahil olmayan ayrı bir ekip, dağıtım testinden sorumlu olmalıdır.
  - Geliştirme ekibi tarafından yapılan sistem testi, sistemdeki hataları keşfetmeye odaklanmalıdır (hata testi). Dağıtım testinin amacı, sistemin gereksinimlerini karşılayıp karşılamadığını ve harici kullanım için yeterince iyi olup olmadığını kontrol etmektir (onaylama testi).

# Gereksinimlere Dayalı Test Etme

- Gereksinim tabanlı test, her bir gereksinimi incelemeyi ve bunun için bir test veya testler geliştirmeyi içerir.
- Mentcare (ZihinSas) sistem gereksinimleri:
  - Bir hastanın belirli bir ilaca alerjisi olduğu biliniyorsa, o ilacın reçetesi sistem kullanıcısına bir uyarı mesajı verilmesiyle sonuçlanacaktır.
  - Bir reçete yazan kişi alerji uyarısını görmezden gelmeyi seçerse, bunun neden göz ardı edildiğine dair bir neden sunmalıdır.

# Gereksinim Testleri

- Bilinen alerjisi olmayan bir hasta kaydı oluşturun. Var olduğu bilinen alerjileri için ilaç yazınız. Sistem tarafından bir uyarı mesajı verilmediğini kontrol edin.
- Bilinen bir alerjisi olan hasta kaydı oluşturun. Hastanın alerjisi olduğu ilacı reçeteye yazın ve uyarının sistem tarafından verilip verilmediğini kontrol edin.
- İki veya daha fazla ilaca karşı alerjisi olan bir hasta kaydı oluşturun. Bu ilaçların her ikisini de ayrı ayrı reçeteye yazın ve her ilaç için doğru uyarının verildiğini kontrol edin.
- Hastanın alerjisi olan iki ilacı reçeteye yazın. İki uyarının doğru şekilde verildiğini kontrol edin.
- Bir uyarı veren ve bu uyarıyı geçersiz kılan bir ilacı reçeteye yazın. Sistemin, kullanıcının uyarının neden reddedildiğini açıklayan bilgi sağlamasını gerektirip gerektirmediğini kontrol edin.

# MentCare için Bir Kullanıcı Hikayesi

Ahmet, ruh sağığı bakımında uzmanlaşmış bir hemşiredir. Onun sorumluluklarından biri, tedavilerinin etkili olduğunu ve hastaların ilaçların yan etkilerini maruz kalmadığını kontrol etmek için hastaları evlerinde ziyaret etmektir.

Ev ziyaretlerinin birinde, ZihinSaS sistemine giriş yapar ve sistemi o güne ait ev ziyaretleri programını yazdırmak ve o gün ziyaret edeceği hastalar hakkında özel bilgiler almak için kullanır. Ahmet bu hastaların kayıtlarını dizüstü bilgisayarına indirme talebinde bulunur ve dizüstü bilgisayarında bu kayıtları şifrelemek için onun anahtar kelimesi istenir.

Onun ziyaret ettiği hastalardan biri, depresyon hastalığı için ilaç kullanan Cem'dir. Cem bu ilacın ona yardımcı olduğunu hisseder fakat ilacın onu geceleri uyutmamak gibi bir yan etkisi olduğuna inanır. Ahmet Cem'in kayıtlarına bakar ve ondan Cem'in kayıtlarının şifresini çözmek onun anahtar sözcüğü ister. Reçetelenen ilacı kontrol eder ve onun yan etkilerini sorgular. Uykusuzluk bilinen bir yan etkidir, bu yüzden Cem'in kayıtlarına problemi not eder ve ona ilaç değişikliği için kliniğı ziyaret etmesini önerir. Cem kabul eder, bu nedenle Ahmet, psikiyatri ile randevu almak için kliniğe geri döndüğünde Cem'i aramak için bir istem girer. Ahmet konsültasyonu sonlandırır ve sistem Cem'in kayıtlarını yeniden şifreler.

Konsültasyonları bittikten sonra, Ahmet kliniğe geri döner ve veri tabanına ziyaret ettiği hastaların kayıtlarını yükler. Sistem, Ahmet için, bilgilerini izlemek ve klinik başvurularını yamak için iletişim halinde olmak zorunda olduğu hastaların bir çağrı listesini oluşturur.

# Senaryoya Göre Test Edilen Özellikler

- Sistemde oturum açarak kimlik doğrulama.
- Belirtilen hasta kayıtlarının bir dizüstü bilgisayara indirilmesi ve yüklenmesi.
- Ev ziyareti zaman çizelgesi.
- Bir mobil cihazda hasta kayıtlarının şifrelenmesi ve şifresinin çözülmesi.
- Kayıt geri getirme ve değiştirme.
- Yan etki bilgilerini tutan ilaç veri tabanı ile bağlantılar kurma.
- Çağrıya hızlı yanıt vermek için bir sistem geliştirme.

# Performans Testi

- Dağıtım testinin bir kısmı, performans ve güvenilirlik gibi bir sistemin ortaya çıkan özelliklerinin test edilmesini içerebilir.
- Testler, sistemin kullanım profilini yansıtmalıdır.
- Performans testleri genellikle, sistem performansı kabul edilemez hale gelene kadar yükün sürekli olarak arttırıldığı bir dizi testin planlanmasını içerir.
- Stres testi, sistemin hata davranışını test etmek için kasıtlı olarak aşırı yüklendiği bir performans testi biçimidir.



# Kullanıcı Testi

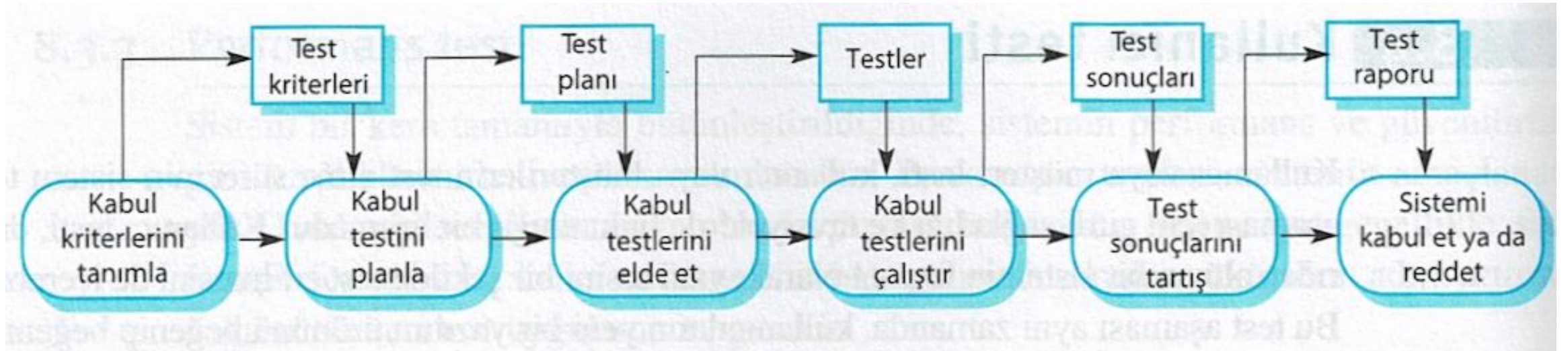
# Kullanıcı Testi

- Kullanıcı veya müşteri testi, kullanıcıların veya müşterilerin sistem testi hakkında girdi ve tavsiye sağladıkları test sürecinde bir aşamadır.
- Kapsamlı sistem ve dağıtım testleri yapıldığında bile kullanıcı testi çok önemlidir.
  - Bunun nedeni, kullanıcının çalışma ortamından gelen etkilerin sistemin güvenilirliği, performansı, kullanılabilirliği ve sağlamlığı üzerinde büyük bir etkiye sahip olmasıdır. Bunlar bir test ortamında çoğaltılamaz.

# Kullanıcı Testinin Türleri

- Alfa testi
  - Yazılımın kullanıcıları, yazılımı geliştiricinin sitesinde test etmek için geliştirme ekibiyle birlikte çalışır.
- Beta testi
  - Kullanıcılara, deney yapmalarına ve sistem geliştiricileriyle keşfettikleri sorunları ortaya çıkarmalarına olanak sağlayan bir yazılım sürümü sunulur.
- Kabul testi
  - Müşteriler, sistemin kabul edilmeye, çalışma ortamlarında uygulanmaya ve geliştirici firmadan teslim alınmaya hazır olup olmadığına karar vermek için sistemi sınarlar.

# Kabul Testi Süreci



# Kabul Testi Sürecindeki Aşamalar

- Kabul kriterlerinin tanımla
- Kabul testini planla
- Kabul testlerini türet
- Kabul testlerini çalıştır
- Test sonuçlarını tartış
- Sistemi reddet/kabul et

# Çevik Yöntemler ve Kabul Testi

- Çevik yöntemlerde, kullanıcı/müşteri geliştirme ekibinin bir parçasıdır ve sistemin kabul edilebilirliğine ilişkin kararlar vermekten sorumludur.
- Testler kullanıcı/müşteri tarafından tanımlanır ve değişiklik yapıldığında otomatik olarak çalıştırılmaları bakımından diğer testlerle entegre edilir.
- Ayrı bir kabul testi süreci yoktur.
- Buradaki temel sorun, yerleşik kullanıcının "tipik" olup olmadığı ve tüm sistem paydaşlarının çıkarlarını temsil edip edemeyeceğidir.

# Anahtar Noktalar

- Test, yalnızca bir programdaki hataların varlığını gösterebilir. Hiç hata kalmadığını gösteremez.
- Geliştirme testi, yazılım geliştirme ekibinin sorumluluğundadır. Müşterilere sunulmadan önce bir sistemin test edilmesinden ayrı bir ekip sorumlu olmalıdır.
- Geliştirme testi, tek tek nesneleri test ettiğiniz birim testini ve ilgili nesne gruplarını test ettiğiniz bileşen testini ve kısmi veya tam sistemleri test ettiğiniz sistem testini içerir.

# Anahtar Noktalar

- Yazılımı test ederken, diğer sistemlerdeki hataları keşfetmede etkili olan test senaryosu türlerini seçmek için deneyim ve kılavuzları kullanarak yazılımı «çökertmeye» çalışmalısınız.
- Mümkün olan her yerde otomatik testler yazmalısınız. Testler, bir sistemde her değişiklik yapıldığında çalıştırılabilen bir programın içine yerleştirilmiştir.
- Önce test geliştirme, testlerin test edilecek koddan önce yazıldığı bir geliştirme yaklaşımıdır.
- Senaryo testi, tipik bir kullanım senaryosu icat etmeyi ve bunu test senaryolarını türetmek için kullanmayı içerir.
- Kabul testi, amacın yazılımın operasyonel ortamda dağıtılacak ve kullanılacak kadar iyi olup olmadığına karar vermek olduğu bir kullanıcı test sürecidir.